



4. Vertex AI 소개 및 개요

```
!pip install datasets torchserve torch-model-archiver torch-workflow-archiver nvgpu
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets) (2.2.2)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.32.2)
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.12/dist-packages (from datasets) (4.67.3)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.6.0)
Requirement already satisfied: multiprocessing<0.70.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (3.6.0)
Requirement already satisfied: fsspec<=2025.3.0, >=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (2025.3.0)
Requirement already satisfied: huggingface-hub>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.24.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from datasets) (26.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from datasets) (6.0.3)
Requirement already satisfied: Pillow in /usr/local/lib/python3.12/dist-packages (from torchserve) (11.3.0)
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from torchserve) (5.9.5)
Requirement already satisfied: wheel in /usr/local/lib/python3.12/dist-packages (from torchserve) (0.46.3)
Requirement already satisfied: enum-compat in /usr/local/lib/python3.12/dist-packages (from torch-model-archiver) (1.9.2)
Requirement already satisfied: ansihtml in /usr/local/lib/python3.12/dist-packages (from torch-model-archiver) (1.9.2)
Requirement already satisfied: arrow in /usr/local/lib/python3.12/dist-packages (from nvgpu) (1.4.0)
Requirement already satisfied: flask in /usr/local/lib/python3.12/dist-packages (from nvgpu) (3.1.3)
Requirement already satisfied: flask-restful in /usr/local/lib/python3.12/dist-packages (from nvgpu) (0.3.10)
Requirement already satisfied: six in /usr/local/lib/python3.12/dist-packages (from nvgpu) (1.17.0)
Requirement already satisfied: termcolor in /usr/local/lib/python3.12/dist-packages (from nvgpu) (3.3.0)
Requirement already satisfied: tabulate in /usr/local/lib/python3.12/dist-packages (from nvgpu) (0.9.0)
Requirement already satisfied: pynvml in /usr/local/lib/python3.12/dist-packages (from nvgpu) (13.0.1)
Requirement already satisfied: aiohttp!=4.0.0a0, !=4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec) (4.0.0)
Requirement already satisfied: hf-xet<2.0.0, >=1.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub) (1.4.3)
Requirement already satisfied: httpx<1, >=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub) (0.23.0)
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingface-hub) (0.24.0)
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub) (4.1.0)
Requirement already satisfied: charset-normalizer<4, >=2 in /usr/local/lib/python3.12/dist-packages (from requests) (3.4.0)
Requirement already satisfied: idna<4, >=2.5 in /usr/local/lib/python3.12/dist-packages (from requests) (3.10.1)
Requirement already satisfied: urllib3<3, >=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests) (2025.1.1)
Requirement already satisfied: python-dateutil>=2.7.0 in /usr/local/lib/python3.12/dist-packages (from arrow) (2.9.0)
Requirement already satisfied: tzdata in /usr/local/lib/python3.12/dist-packages (from arrow) (2026.1)
Requirement already satisfied: blinker>=1.9.0 in /usr/local/lib/python3.12/dist-packages (from flask) (1.9.0)
Requirement already satisfied: click>=8.1.3 in /usr/local/lib/python3.12/dist-packages (from flask) (8.3.0)
Requirement already satisfied: itsdangerous>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from flask) (3.0.0)
Requirement already satisfied: jinja2>=3.1.2 in /usr/local/lib/python3.12/dist-packages (from flask) (3.1.2)
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from flask) (3.0.2)
Requirement already satisfied: werkzeug>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from flask) (3.1.0)
Requirement already satisfied: aniso8601>=0.82 in /usr/local/lib/python3.12/dist-packages (from flask-restful) (0.82)
Requirement already satisfied: pytz in /usr/local/lib/python3.12/dist-packages (from flask-restful) (2025.2)
Requirement already satisfied: nvidia-ml-py>=12.0.0 in /usr/local/lib/python3.12/dist-packages (from pynvml) (12.0.0)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp) (2.5.0)
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp) (25.1.0)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp) (1.1.1)
Requirement already satisfied: multidict<7.0, >=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp) (6.0.0)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp) (0.2.0)
Requirement already satisfied: yarl<2.0, >=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp) (1.17.0)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx) (4.7.0)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx) (1.0.7)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore) (0.16.0)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer) (1.3.0)
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer) (13.7.1)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer) (0.0.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich) (3.0.0)
Requirement already satisfied: pygments<3.0.0, >=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich) (2.18.0)
Requirement already satisfied: mdurl<=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py) (0.1.2)
```

```
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import seaborn as sns

from torch.utils.data import DataLoader
from torch.optim import AdamW
from transformers import BatchEncoding, BertTokenizer, BertForSequenceClassification
from sklearn.metrics import confusion_matrix
from datasets import load_dataset
from tqdm import tqdm
from typing import TypedDict
import matplotlib.pyplot as plt
```

```
/usr/local/lib/python3.12/dist-packages/torch/cuda/__init__.py:65: FutureWarning: The pynvml package is deprecated
import pynvml # type: ignore[import]
```

```
dataset = load_dataset("ag_news")
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=4)
optimizer = AdamW(model.parameters(), lr=5e-5)
criterion = torch.nn.CrossEntropyLoss()

class DatasetItem(TypedDict):
    text: str
    label: str

def preprocess_data(dataset_item: DatasetItem) -> dict[str, torch.Tensor]:
    return tokenizer(dataset_item["text"], truncation=True, padding="max_length", return_tensors="pt")

train_dataset = dataset["train"].select(range(1200)).map(preprocess_data, batched=True)
test_dataset = dataset["test"].select(range(800)).map(preprocess_data, batched=True)

train_dataset.set_format("torch", columns=["input_ids", "attention_mask", "label"])
test_dataset.set_format("torch", columns=["input_ids", "attention_mask", "label"])

train_loader = DataLoader(train_dataset, batch_size=8, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=8, shuffle=False)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens)
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
```

```
README.md: 8.07k/? [00:00<00:00, 184kB/s]
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limit.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a
```

```
data/train-00000-of-00001.parquet: 100% 18.6M/18.6M [00:02<00:00, 28.7MB/s]
```

```
data/test-00000-of-00001.parquet: 100% 1.23M/1.23M [00:00<00:00, 2.87MB/s]
```

```
Generating train split: 100% 120000/120000 [00:00<00:00, 338047.09 examples/s]
```

```
Generating test split: 100% 7600/7600 [00:00<00:00, 120176.55 examples/s]
```

```
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 2.39kB/s]
```

```
vocab.txt: 232k/? [00:00<00:00, 5.06MB/s]
```

```
tokenizer.json: 466k/? [00:00<00:00, 3.80MB/s]
```

```
config.json: 100% 570/570 [00:00<00:00, 18.0kB/s]
```

```
model.safetensors: 100% 440M/440M [00:02<00:00, 194MB/s]
```

```
Loading weights: 100% 199/199 [00:01<00:00, 182.59it/s, Materializing param=bert.pooler.dense.weight]
```

```
BertForSequenceClassification LOAD REPORT from: bert-base-uncased
```

Key	Status
cls.predictions.transform.LayerNorm.weight	UNEXPECTED
cls.predictions.bias	UNEXPECTED
cls.seq_relationship.weight	UNEXPECTED
cls.predictions.transform.dense.bias	UNEXPECTED
cls.predictions.transform.dense.weight	UNEXPECTED
cls.predictions.transform.LayerNorm.bias	UNEXPECTED
cls.seq_relationship.bias	UNEXPECTED
classifier.bias	MISSING
classifier.weight	MISSING

```
Notes:
```

```
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING :those params were newly initialized because missing from the checkpoint. Consider training on your data
```

```
Map: 100% 1200/1200 [00:02<00:00, 508.82 examples/s]
```

```
Map: 100% 800/800 [00:01<00:00, 733.43 examples/s]
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
```

```
num_epochs = 3
losses: list[float] = []
```

```
for epoch in range(num_epochs):
```

```

model.train()
total_loss = 0
for batch in tqdm(train_loader, desc=f"Epoch {epoch + 1}"):
    inputs = {key: batch[key].to(device) for key in batch}
    labels = inputs.pop("label")
    outputs = model(**inputs, labels=labels)
    loss = outputs.loss
    total_loss += loss.item()
    losses.append(loss.item())

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

average_loss = total_loss / len(train_loader)
print(f"Epoch {epoch + 1}, Average Loss: {average_loss}")

```

```

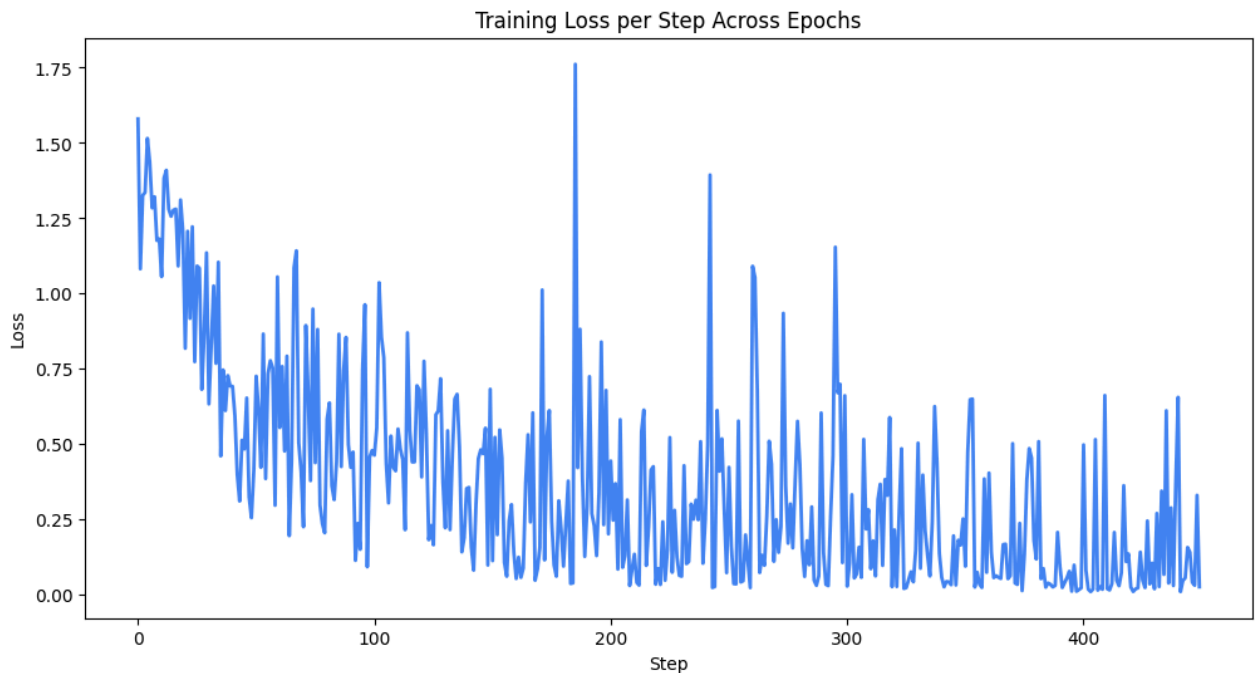
Epoch 1: 100%|██████████| 150/150 [01:43<00:00, 1.45it/s]
Epoch 1, Average Loss: 0.6614672436813513
Epoch 2: 100%|██████████| 150/150 [01:48<00:00, 1.39it/s]
Epoch 2, Average Loss: 0.30290428844590983
Epoch 3: 100%|██████████| 150/150 [01:51<00:00, 1.35it/s]Epoch 3, Average Loss: 0.16283869226152697

```

```

plt.figure(figsize=(12, 6))
plt.plot(losses, color="#4285f4", linewidth=2)
plt.xlabel("Step")
plt.ylabel("Loss")
plt.title("Training Loss per Step Across Epochs")
plt.show()

```



```

model.eval()
correct = 0
total = 0

with torch.no_grad():
    for batch in tqdm(test_loader, desc="Evaluating"):
        inputs = {key: batch[key].to(device) for key in batch}
        labels = inputs.pop("label")
        outputs = model(**inputs, labels=labels)
        logits = outputs.logits
        predicted_labels = torch.argmax(logits, dim=1)
        correct += (predicted_labels == labels).sum().item()
        total += labels.size(0)

accuracy = correct / total

print("")
print(f"Test Accuracy: {accuracy * 100:.2f}%")

```

```

Evaluating: 100%|██████████| 100/100 [00:23<00:00, 4.30it/s]
Test Accuracy: 83.75%

```

```
test_input = "[Official] 'Legendary Coach Resigns → Appoints New Commander' Suwon Completes Coaching Staff... Sc
test_input_processed = tokenizer(test_input, truncation=True, padding="max_length", return_tensors="pt").to(devi
logits = model(**test_input_processed).logits
print(logits)
predicted_labels = torch.argmax(logits, dim=1)

labeling_mapper = ["world", "sports", "business", "sci/tech"]
print(labeling_mapper[predicted_labels[0]])

tensor([[ 2.2219,  1.5693, -2.2677, -2.6118]], device='cuda:0',
      grad_fn=<AddmmBackward0>)
world
```

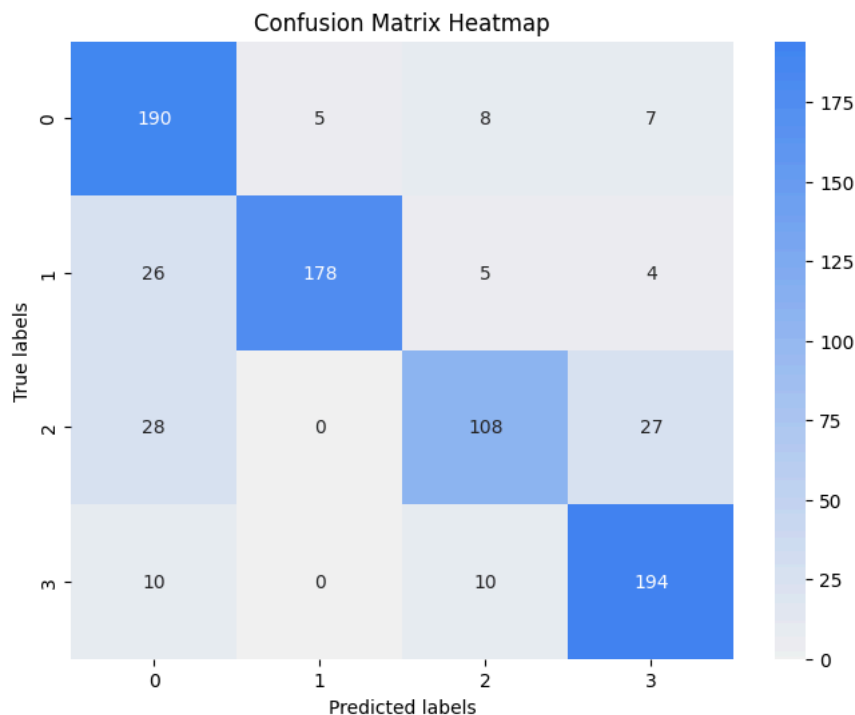
```
all_predictions: list[int] = []
all_labels: list[int] = []

with torch.no_grad():
    for batch in tqdm(test_loader, desc="Evaluating"):
        inputs = {key: batch[key].to(device) for key in batch}
        labels = inputs.pop("label")
        outputs = model(**inputs)
        logits = outputs.logits
        predicted_labels = torch.argmax(logits, dim=1)

        all_predictions.extend(predicted_labels.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())
```

Evaluating: 100%|██████████| 100/100 [00:23<00:00, 4.32it/s]

```
conf_matrix = confusion_matrix(all_labels, all_predictions)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt="g", cmap=sns.light_palette("#4285f4", as_cmap=True))
plt.xlabel("Predicted labels")
plt.ylabel("True labels")
plt.title("Confusion Matrix Heatmap")
plt.show()
```



▼ TorchServe Archive

```
# 모델 체크포인트 저장.
model_save_path = "model.pth"
torch.save(model.state_dict(), model_save_path)
```

```
%%writefile handler.py
import json
import logging
```

```

import torch
from ts.context import Context
from ts.torch_handler.base_handler import BaseHandler
from transformers import BatchEncoding, BertTokenizer, BertForSequenceClassification

logging.basicConfig(level=logging.INFO)

class ModelHandler(BaseHandler):
    def __init__(self):
        self.initialized = False
        self.tokenizer = None
        self.model = None

    def initialize(self, context: Context):
        properties = context.system_properties
        model_dir = properties.get("model_dir")
        self.initialized = True
        self.tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
        self.model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=4)
        model_path = model_dir + "/model.pth"
        self.model.load_state_dict(torch.load(model_path))
        self.model.to(torch.device("cuda" if torch.cuda.is_available() else "cpu"))
        self.model.eval()

    def preprocess(self, texts: list[str]) -> BatchEncoding:
        logging.info("preprocess", texts)

        inputs = self.tokenizer(texts, truncation=True, padding=True, max_length=512, return_tensors="pt")
        device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        return inputs.to(device)

    def inference(self, input_batch: BatchEncoding) -> torch.Tensor:
        with torch.no_grad():
            outputs = self.model(**input_batch)
            logging.info("inference", outputs)
            return outputs.logits

    def postprocess(self, inference_output: torch.Tensor) -> list[dict[str, float]]:
        logging.info("postprocess", inference_output)

        probabilities = torch.nn.functional.softmax(inference_output, dim=1)
        return [{"label": int(torch.argmax(prob)), "probability": float(prob.max())} for prob in probabilities]

```

Writing handler.py

bert vocab 파일 (아티팩트)

```
!wget https://raw.githubusercontent.com/microsoft/SDNet/master/bert_vocab_files/bert-base-uncased-vocab.txt \
-O bert-base-uncased-vocab.txt
```

```
--2026-04-21 02:13:13-- https://raw.githubusercontent.com/microsoft/SDNet/master/bert_vocab_files/bert-base-uncased-vocab.txt
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, 185.199.111.133
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 231508 (226K) [text/plain]
Saving to: 'bert-base-uncased-vocab.txt'
```

```
bert-base-uncased-v 100%[=====] 226.08K --.-KB/s in 0.02s
```

```
2026-04-21 02:13:13 (14.4 MB/s) - 'bert-base-uncased-vocab.txt' saved [231508/231508]
```

```
# Torch serve archiver의 최종 파일 저장 위치.
# .mar 확장자 파일을 생성하여 패키징 하게 된다.
```

```
!mkdir -p model-store
```

▼ Register to Vertex AI

```
!torch-model-archiver \
--model-name model \
--version 1.0 \
--serialized-file model.pth \
--handler ./handler.py \
--extra-files "bert-base-uncased-vocab.txt" \
--export-path model-store \
-f
```

```
# from google.colab import auth
# auth.authenticate_user()
```

```
from google.cloud import storage, aiplatform
from google.oauth2 import service_account
```

```
credentials = service_account.Credentials.from_service_account_file("credentials.json")
```

```
PROJECT_ID = "project-cc4df86d-eb34-4f99-832"
LOCATION = "asia-northeast3"
BUCKET_NAME = "mlops-quicklab-5" # @param {type: "s
MODEL_FILE_NAME = "model.mar" # @param {
```

PROJECT_ID

project-cc4df86d-eb34-4f

LOCATION

asia-northeast3

BUCKET_NAME

mlops-quicklab-5

MODEL_FILE_NAME

model.mar

[코드 숨기기](#)

```
storage_client = storage.Client(credentials=credentials)
bucket = storage_client.bucket(BUCKET_NAME)
```

```
def upload_blob(source_file_name: str, destination_blob_name: str) -> None:
    blob = bucket.blob(destination_blob_name)
    blob.upload_from_filename(source_file_name)
    print(f"File {source_file_name} uploaded to {destination_blob_name}.")
```

```
upload_blob("model-store/model.mar", f"models/{MODEL_FILE_NAME}")
```

```
File model-store/model.mar uploaded to models/model.mar.
```

```
aiplatform.init(
    project=PROJECT_ID,
    location=LOCATION,
    credentials=credentials,
)
```

```
model_path = f"gs://{BUCKET_NAME}/models"
```

```
registry_model = aiplatform.Model.upload(
    display_name="AG News Classification",
    artifact_uri=model_path,
    serving_container_image_uri="asia-northeast3-docker.pkg.dev/project-cc4df86d-eb34-4f99-832/mlops-cicd/train",
    is_default_version=True,
    version_aliases=["v1"],
    version_description="A news category classification model",
    serving_container_predict_route="/predictions/model",
    serving_container_health_route="/ping",
)
```

```
DEPLOY_COMPUTE = "n1-standard-2"
DEPLOY_ACCELERATOR = "NVIDIA_TESLA_T4"
```

```
endpoint = aiplatform.Endpoint.create(
    display_name="ag-news-category-classification",
    project=PROJECT_ID,
    location=LOCATION,
)
```

```
from google.cloud import aiplatform
```

```
# 필요 시 이미 앞에서 init 했더라도 한 번 더 맞춰줌
```

```
aiplatform.init(
    project="project-cc4df86d-eb34-4f99-832",
    location="asia-northeast3",
)
```

```
# 모델 아티팩트 경로
```

```
model_path = f"gs://{BUCKET_NAME}/models"
```

```
# 새 x86(amd64) 이미지로 모델 다시 업로드
```

```
registry_model = aiplatform.Model.upload(
    display_name="AG News Classification",
    artifact_uri=model_path,
    serving_container_image_uri="asia-northeast3-docker.pkg.dev/project-cc4df86d-eb34-4f99-832/mlops-cicd/train",
    is_default_version=True,
```

```
version_aliases=["v2"],
version_description="A news category classification model",
serving_container_predict_route="/predictions/model",
serving_container_health_route="/ping",
sync=True,
)

# 배포
deployment = registry_model.deploy(
    endpoint=endpoint,
    machine_type=DEPLOY_COMPUTE,
    min_replica_count=1,
    max_replica_count=1,
    traffic_percentage=100,
    sync=True,
)

print("Model uploaded:", registry_model.resource_name)
print("Deployment done:", deployment)
```

```
Model uploaded: projects/991235266460/locations/asia-northeast3/models/8820647315879493632
Deployment done: <google.cloud.aiplatform.models.Endpoint object at 0x7ec2cf623590>
resource name: projects/991235266460/locations/asia-northeast3/endpoints/3962897192225603584
```

```
endpoint.predict(instances=[
    "OpenAI releases AI video generator Sora to all customers"
])
```

✓ Clean up

```
endpoint.undeploy_all()
endpoint.delete()
```

```
registry_model.delete()
```